AXI4-Lite Master - Slave Communication System

1. Overview

This project implements a **complete AXI4-Lite communication system** consisting of a custom **AXI4-Lite Master** and **AXI4-Lite Slave** designed in Verilog RTL.

The system includes:

- A **configurable AXI4-Lite master controller**
- A **register-based AXI4-Lite slave peripheral**
- Full support for **read and write channels**
- **Finite State Machines (FSMs)** for transaction management
- **Byte-enable write support using WSTRB**
- **Protocol-compliant VALID/READY handshakes**
- **Error handling using AXI response signals**

The slave module implements a **register file memory block** where certain address regions are configured as **read-only or write-only** to demonstrate protocol error responses.

2. System Architecture

The system consists of two primary modules:

AXI4-Lite Master

Generates read and write transactions.

AXI4-Lite Slave

Receives transactions and performs memory operations.

Data transfer occurs using the **five AXI4-Lite channels**:

1. Write Address Channel
2. Write Data Channel
3. Write Response Channel
4. Read Address Channel
5. Read Data Channel

These channels operate using independent **VALID-READY handshaking**, which allows flexible transaction timing.

AXI4-Lite_CustomPLPeripheralCon…

3. AXI4-Lite Interface Signals

Clock and Reset

Signal	Direction	Description
ACLK	Input	Global AXI clock
ARESETn	Input	Active-low reset

4. Write Address Channel

Signal	Direction	Description
AWADDR	Master → Slave	Write address
AWPROT	Master → Slave	Protection attributes
AWVALID	Master → Slave	Address valid
AWREADY	Slave → Master	Address accepted

5. Write Data Channel

Signal	Direction	Description
WDATA	Master → Slave	Write data
WSTRB	Master → Slave	Byte enables
WVALID	Master → Slave	Write data valid
WREADY	Slave → Master	Data accepted

6. Write Response Channel

Signal	Direction	Description
BRESP	Slave → Master	Write response
BVALID	Slave → Master	Response valid
BREADY	Master → Slave	Response accepted

7. Read Address Channel

Signal	Direction	Description
ARADDR	Master → Slave	Read address
ARPROT	Master → Slave	Protection attributes
ARVALID	Master → Slave	Address valid
ARREADY	Slave → Master	Address accepted

8. Read Data Channel

Signal	Direction	Description
RDATA	Slave → Master	Read data
RRESP	Slave → Master	Read response
RVALID	Slave → Master	Data valid
RREADY	Master → Slave	Data accepted

9. AXI Response Encoding

Response	Encoding	Description
OKAY	00	Successful transaction
EXOKAY	01	Exclusive access success
SLVERR	10	Slave generated error
DECERR	11	Decode error (invalid address)

10. Slave Memory Organization

The slave module implements a **register file with configurable depth**.

Example configuration:

```
MEM_DEPTH = 16
DATA_WIDTH = 32 bits
```

Each memory location stores **32-bit data**.

AXI uses byte addressing. Internal memory uses word addressing.
Word Address = AWADDR[5:2]/ARADDR[5:2]
Valid byte address range: 0x00 to 0x3F (0 to 63 decimal).
All accesses must be 32-bit aligned (AWADDR[1:0]/ARADDR[1:0] == 2'b00). Unaligned access generates SLVERR.
.

Special Address Regions

Byte Address Range	Word Index	Access Type	Description
0x00 – 0x24	0 – 9	Read/Write	Normal registers
0x28 – 0x30	10 – 12	Read-Only	Status registers
0x34 – 0x38	13 – 14	Write-Only	Command registers
0x3C	15	Read/Write	Reserved / normal
Others	-	Invalid	DECERR

11. Write Transaction Sequence

A write transaction occurs as follows:

1. Master asserts **AWVALID** with write address.
2. Master asserts **WVALID** with write data and byte strobes.
3. Slave asserts **AWREADY** to accept address.
4. Slave asserts **WREADY** to accept data.
5. Address and data may arrive in **any order**.
6. Slave performs the memory write.
7. Slave asserts **BVALID** with response.
8. Master asserts **BREADY**.
9. Transaction completes.

This handshake behavior follows the AXI rule that **a transfer occurs when VALID and READY are high on the same clock edge**.

AXI4-Lite_CustomPLPeripheralCon…

12. Read Transaction Sequence

A read transaction occurs as follows:

1. Master asserts **ARVALID** with read address.
2. Slave asserts **ARREADY**.
3. Slave retrieves data from register file.
4. Slave asserts **RVALID** with read data.
5. Master asserts **RREADY**.
6. Transaction completes.

13. Write Channel Finite State Machine (Master)

The master write logic is controlled by a **five-state FSM**.

States

State	Description
W_IDLE	Wait for write request
W_BOTH	Send address and data
W_ADDR	Waiting for address handshake
W_DATA	Waiting for data handshake
W_RESP	Waiting for write response

Operation

1. In **W_IDLE**, the master waits for a write request.
2. In **W_BOTH**, both address and data are sent.
3. If one handshake completes earlier, the FSM moves to **W_ADDR** or **W_DATA**.
4. Once both handshakes finish, the master waits for **BVALID**.
5. When the response is accepted, the FSM returns to **W_IDLE**.

14. Read Channel Finite State Machine (Master)

The master read logic uses a **three-state FSM**.

States

State	Description
R_IDLE	Wait for read request
R_ADDR	Wait for slave ready
R_DATA	Wait for read data

Operation

1. In **R_IDLE**, the master waits for a read request.
2. Slave waits for ARVALID
3. After the slave accepts the address, the FSM waits for **RVALID**.
4. The data is captured and the FSM returns to **R_IDLE**.

15. Slave Write FSM

The slave write logic supports **independent address and data arrival**.

States

State	Description
W_IDLE	Ready to accept address and data
W_BOTH	Waiting for both address and data
W_ADDR	Address received first
W_DATA	Data received first
W_RESP	Generate write response

Features

- Supports **decoupled address and data channels**
- Implements **byte-enable writes**
- Handles **invalid address errors**
- Prevents writes to **read-only registers**

16. Slave Read FSM

The slave read logic is implemented using a **two-state FSM**.

States

State	Description
R_IDLE	Wait for read request
R_DATA	Send read data

Features

- Supports **address validation**
- Generates **error responses for write-only regions**
- Returns register data to the master

17. Byte Enable (WSTRB) Support

The design supports **partial word updates** using the WSTRB signal.

WSTRB Bit	Affected Data
WSTRB[0]	Bits [7:0]
WSTRB[1]	Bits [15:8]
WSTRB[2]	Bits [23:16]
WSTRB[3]	Bits [31:24]

This allows selective byte updates without overwriting the entire register.

18. Parallel Operation

The **read and write FSMs operate independently**, allowing simultaneous transactions.

19. Design Assumptions

The following assumptions are made in the implementation of the AXI4-Lite master and slave modules:

- All AXI interface signals operate synchronously with the **ACLK** clock signal.
- The reset signal **ARESETn** is an active-low reset that initializes the internal FSMs and interface signals.
- Address accesses are assumed to be **word-aligned**.
- The design supports **only one active transaction at a time**.
- The memory implemented in the slave module is limited to **MEM_DEPTH registers**.
- The master must follow the AXI protocol rule that **VALID signals remain asserted until the corresponding READY signal is observed**.
- Read and write channels operate **independently and concurrently**, as defined by the AXI4-Lite specification.

20. Error Handling Behavior

The slave module generates AXI responses based on the type of access performed.

Condition	**BRESP**
Unaligned address	SLVERR
Address out of range	DECERR
Write to RO region and Read to WO region	SLVERR
Valid write	OKAY

These responses allow the master to determine whether the transaction completed successfully or encountered an error.

21. Reset Behavior

When the reset signal **ARESETn** is asserted low:

- All interface VALID signals are cleared.
- READY signals are reset to their default state.
- All finite state machines return to their **IDLE state**.
- Internal registers and memory elements are reset to zero.

This ensures the AXI interface starts from a known state when the system is initialized.